

Отчет по #22

«Диалог подтверждения при запуске лекции с неохранными изменениями»

Задание:

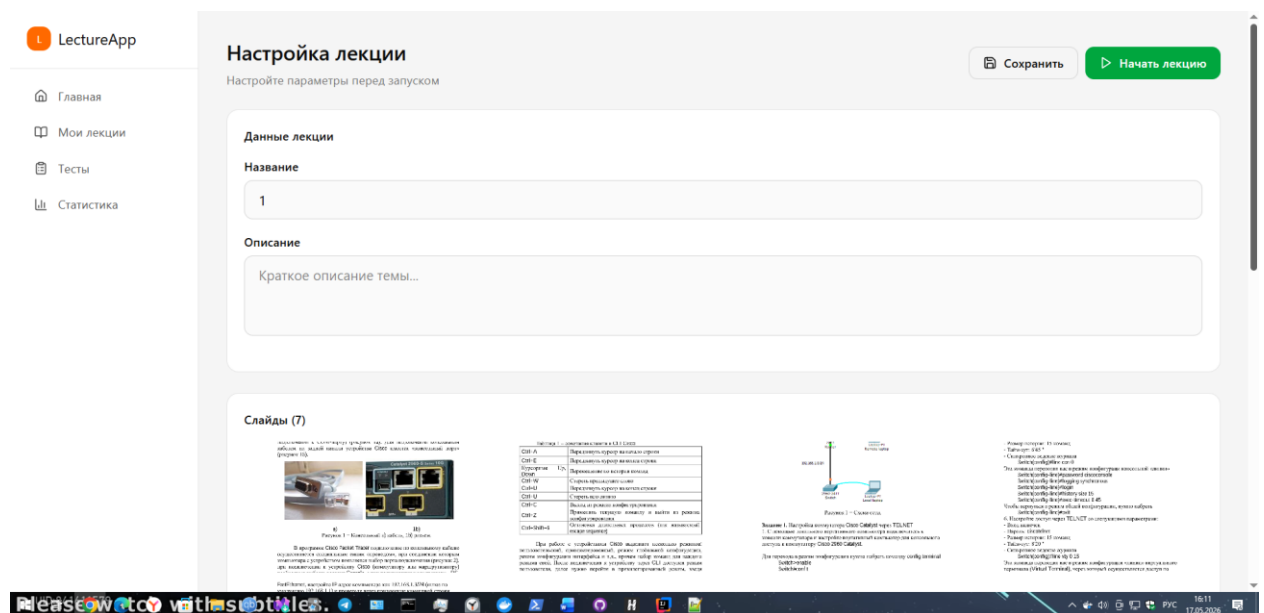
Если что-то меняется в настройках лекции, то перед запуском без сохранения показать плашку подтверждения (сохранить или запустить без них).

- react-app (LectureSettingsPage, LivePresentationPage)

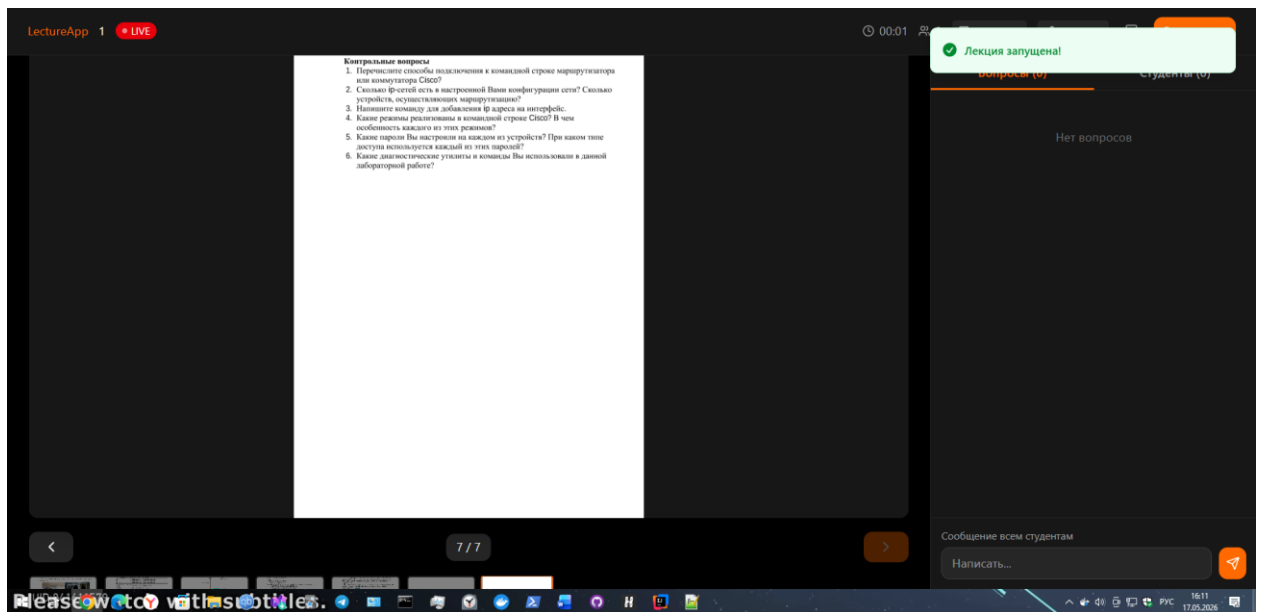
Задачи для:

- **Frontend - Design:** Дизайн модального окна подтверждения
- **React:** Отслеживание изменений в settingsPage, показ модала перед запуском
- **Testing:** Проверить что модал показывается при наличии несохраненных изменений

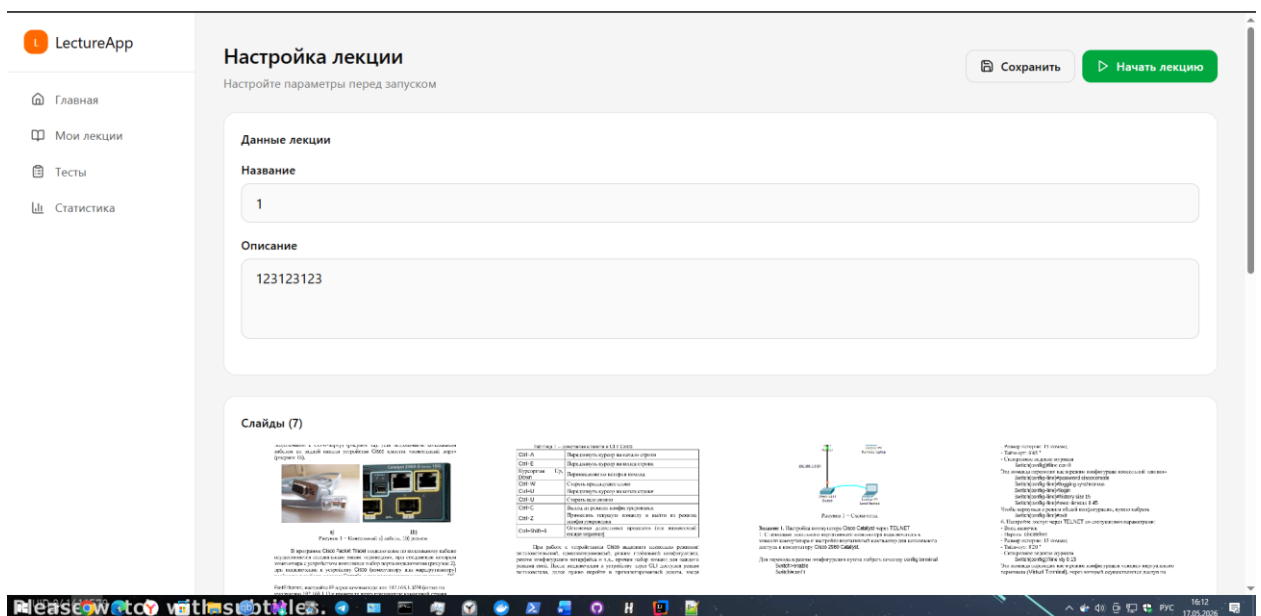
Внесены изменения в LectureSettingsPage которые позволяют отображать модельное окно при внесении изменений в презентацию отображаемой перед её запуском если не было сохранения:



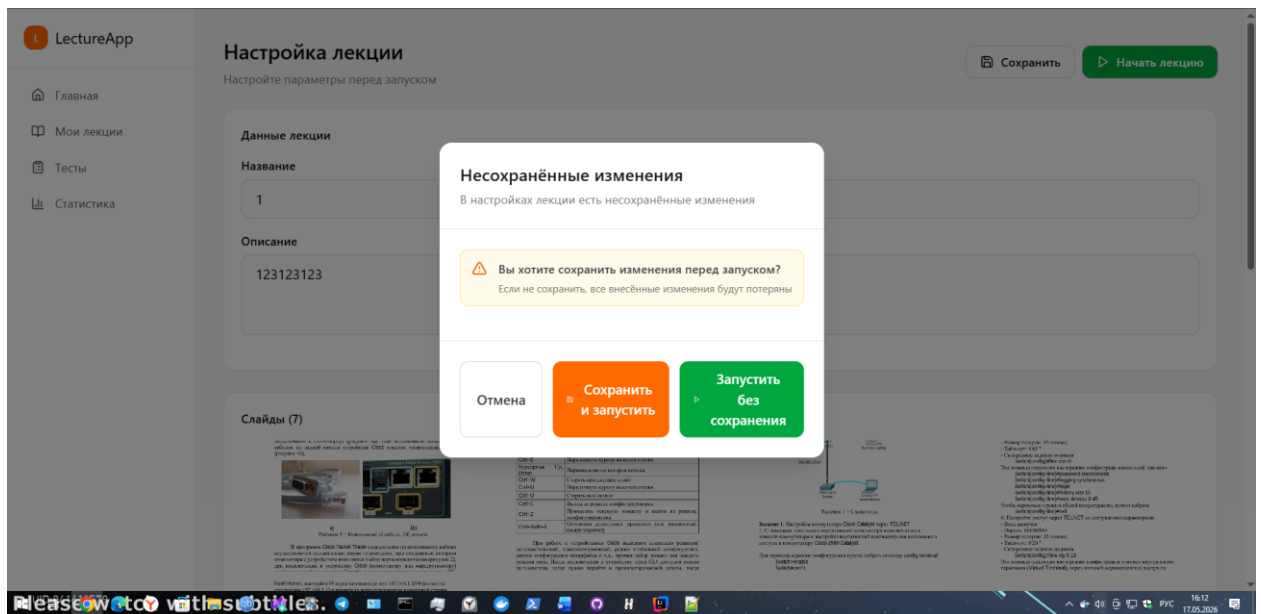
Изначальный вид лекции



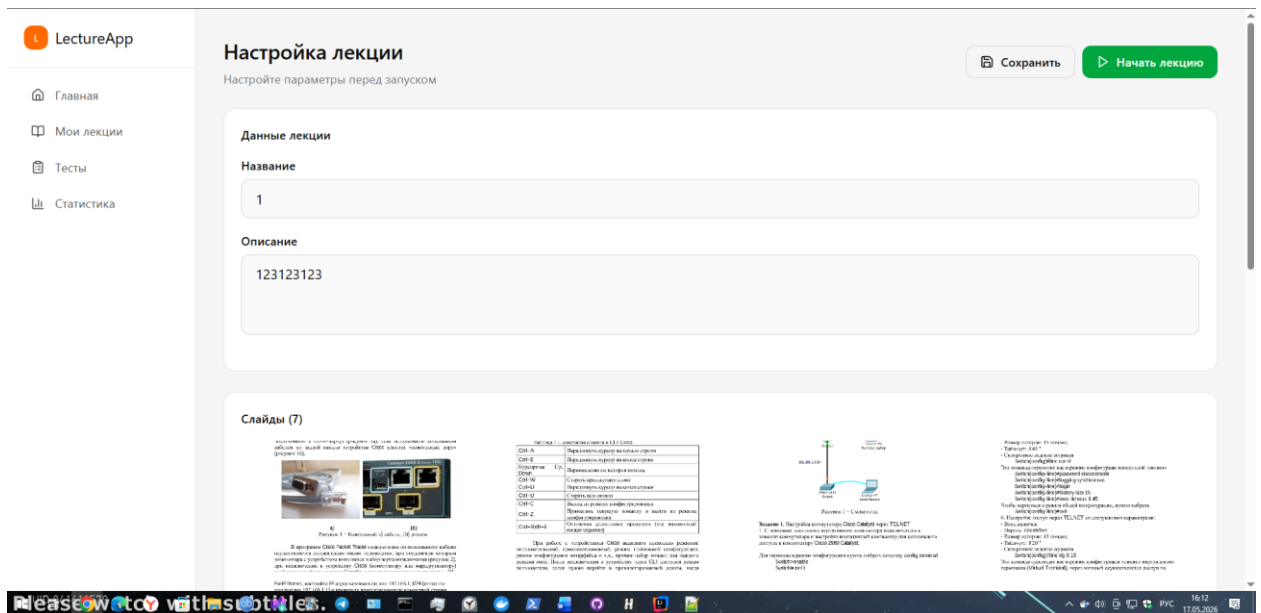
Запуск лекции без изменений

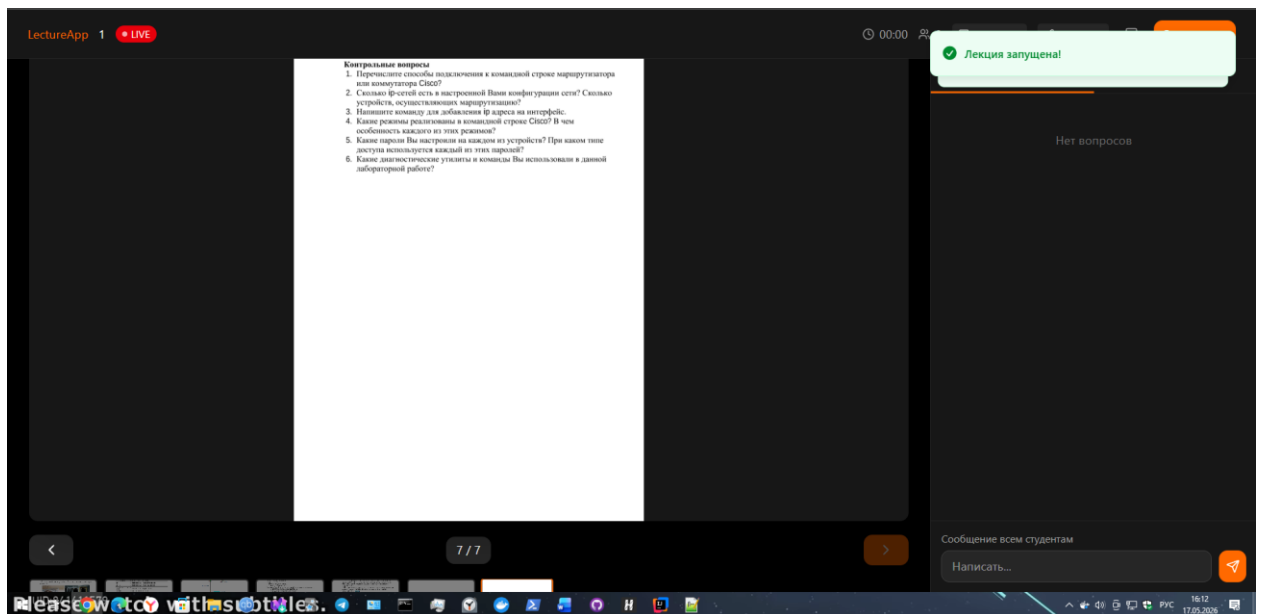


Внесены изменения

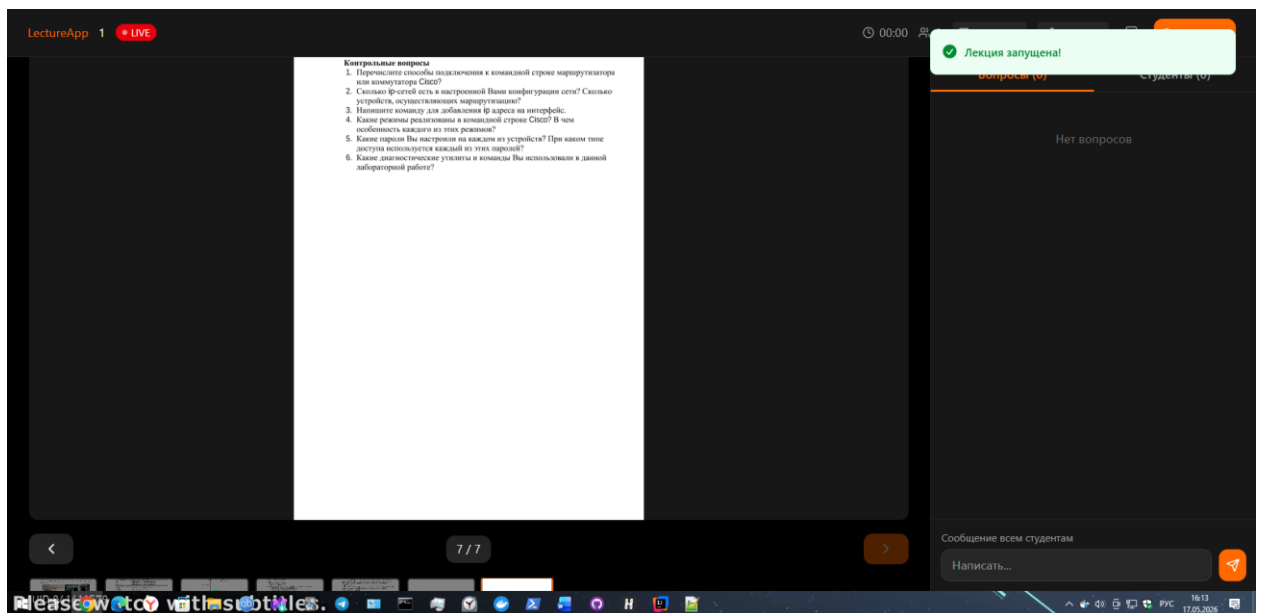


Отображаемое окно перед запуском без сохранения изменений





сохранить и запустить



запустить без сохранения

Для запуска установить node.js на ПК

Cmd:

cd C:\Users\Seren\Documents\StudentPJ\TechDolg\lecturer-assistant\react-app

npm install

npm run build

dir dist

npm run dev

Весь лог cmd:

```
C:\Users\Seren\Documents\StudentPJ\TechDolg\lecturer-assistant\react-app>dir dist
```

Том в устройстве C не имеет метки.

Серийный номер тома: 7282-7945

Содержимое папки C:\Users\Seren\Documents\StudentPJ\TechDolg\lecturer-assistant\react-app\dist

```
17.05.2026 14:56 <DIR>      .
17.05.2026 14:56 <DIR>      ..
17.05.2026 14:56 <DIR>      assets
17.05.2026 14:56          460 index.html

      1 файлов      460 байт

      3 папок  2 264 309 760 байт свободно
```

```
C:\Users\Seren\Documents\StudentPJ\TechDolg\lecturer-assistant\react-app>cd
C:\Users\Seren\Documents\StudentPJ\TechDolg\lecturer-assistant\react-app
```

```
C:\Users\Seren\Documents\StudentPJ\TechDolg\lecturer-assistant\react-app>npm install
```

up to date, audited 309 packages in 3s

40 packages are looking for funding

run `npm fund` for details

4 vulnerabilities (2 moderate, 2 high)

To address issues that do not require attention, run:

npm audit fix

To address all issues, run:

```
npm audit fix --force
```

Run `npm audit` for details.

```
C:\Users\Seren\Documents\StudentPJ\TechDolg\lecturer-assistant\react-app>npm run build
```

```
> @figma/my-make-file@0.0.1 build
```

```
> vite build
```

```
vite v6.3.5 building for production...
```

```
✓ 1653 modules transformed.
```

```
[plugin vite:reporter]
```

```
(!) C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-app/src/app/api/client.ts is dynamically imported by C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-app/src/app/api/quiz.api.ts, C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-app/src/app/api/quiz.api.ts, C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-app/src/app/api/quiz.api.ts but also statically imported by C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-app/src/app/api/analytics.api.ts, C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-app/src/app/api/lecture.api.ts, C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-app/src/app/api/quiz.api.ts, C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-app/src/pages/HomePage.tsx, C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-app/src/pages/LectureSettingsPage.tsx, C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-app/src/pages/LivePresentationPage.tsx, C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-app/src/pages/MyLecturesPage.tsx, C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-app/src/pages/ProjectionPage.tsx, C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-app/src/pages/SlideManagerPage.tsx, C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-
```

app/src/pages/StatisticsPage.tsx, C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-app/src/pages/TestsPage.tsx,
C:/Users/Seren/Documents/StudentPJ/TechDolg/lecturer-assistant/react-app/src/pages/UploadPresentationPage.tsx, dynamic import will not move module into another chunk.

dist/index.html 0.46 kB | gzip: 0.32 kB

dist/assets/index-CWNFWPud.css 107.38 kB | gzip: 17.37 kB

dist/assets/index-C9f4QByd.js 492.88 kB | gzip: 144.78 kB

✓ built in 7.08s

C:\Users\Seren\Documents\StudentPJ\TechDolg\lecturer-assistant\react-app>dir dist

Том в устройстве C не имеет метки.

Серийный номер тома: 7282-7945

Содержимое папки C:\Users\Seren\Documents\StudentPJ\TechDolg\lecturer-assistant\react-app\dist

```
17.05.2026 16:05 <DIR>      .
17.05.2026 16:05 <DIR>      ..
17.05.2026 16:05 <DIR>      assets
17.05.2026 16:05          460 index.html
          1 файлов      460 байт
          3 папок  2 308 235 264 байт свободно
```

C:\Users\Seren\Documents\StudentPJ\TechDolg\lecturer-assistant\react-app>npm run dev

> @figma/my-make-file@0.0.1 dev

> vite --host 127.0.0.1

VITE v6.3.5 ready in 806 ms

→ Local: <http://127.0.0.1:5173/>

→ press h + enter to show help

Текущий файл docker-compose.yml изменен так как в изначальном нет подвязки к web
name: digital-lecturer

services:

postgres:

image: postgres:15-alpine

container_name: postgres_db

environment:

POSTGRES_USER: user

POSTGRES_PASSWORD: password

POSTGRES_DB: root_db

ports:

- "5432:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

- ./postgres-init/init.sql:/docker-entrypoint-initdb.d/init.sql

networks:

- app-network

restart: unless-stopped

content-service:

build: ./content-service

container_name: content_service

ports:

- "8081:8081"

environment:

- SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/content_db
- SPRING_DATASOURCE_USERNAME=user
- SPRING_DATASOURCE_PASSWORD=password

depends_on:

postgres:

condition: service_started

networks:

- app-network

lecture-broadcasting-service:

build: ./lecture-broadcasting-service

container_name: lecture_broadcasting_service

ports:

- "8082:8082"

environment:

- SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/broadcasting_db
- SPRING_DATASOURCE_USERNAME=user
- SPRING_DATASOURCE_PASSWORD=password
- TELEGRAM_BOT_TOKEN=\${TELEGRAM_BOT_TOKEN}
- TELEGRAM_BOT_USERNAME=\${TELEGRAM_BOT_USERNAME}
- CONTENT_SERVICE_URL=http://content-service:8081
- ANALYTICS_SERVICE_URL=http://analytics-service:8084
- QUIZ_SERVICE_URL=http://quiz-service:8083

depends_on:

postgres:

condition: service_started

networks:

- app-network

quiz-service:

build: ./quiz-service

container_name: quiz_service

ports:

- "8083:8083"

environment:

- SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/quiz_db
- SPRING_DATASOURCE_USERNAME=user
- SPRING_DATASOURCE_PASSWORD=password

depends_on:

postgres:

condition: service_started

networks:

- app-network

analytics-service:

build: ./analytics-service

container_name: analytics_service

ports:

- "8084:8084"

environment:

- SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/analytics_db
- SPRING_DATASOURCE_USERNAME=user
- SPRING_DATASOURCE_PASSWORD=password

depends_on:

postgres:

condition: service_started

networks:

- app-network

admin-gateway:

build: ./admin-gateway

container_name: admin_gateway

ports:

- "8080:8080"

environment:

- CONTENT_SERVICE_URL=http://content-service:8081
- LECTURE_BROADCASTING_SERVICE_URL=http://lecture-broadcasting-service:8082
- QUIZ_SERVICE_URL=http://quiz-service:8083
- ANALYTICS_SERVICE_URL=http://analytics-service:8084

depends_on:

- content-service
- lecture-broadcasting-service
- quiz-service
- analytics-service

networks:

- app-network

Фронтенд - используем готовый образ nginx с локальной сборкой

frontend:

image: nginx:alpine

container_name: react_frontend

ports:

- "3000:3000"

volumes:

- ./react-app/dist:/usr/share/nginx/html:ro

depends_on:

- admin-gateway

networks:

- app-network

restart: unless-stopped

networks:

app-network:

driver: bridge

volumes:

postgres_data:

dockerFile ReaktUp:

react-app/Dockerfile

FROM node:18-alpine AS builder

WORKDIR /app

Копируем package.json и package-lock.json

COPY package*.json ./

Устанавливаем зависимости

RUN npm ci --only=production

Копируем исходники

COPY . .

Сборка приложения

RUN npm run build

Этап production с nginx

FROM nginx:alpine

Копируем сборку в nginx

COPY --from=builder /app/build /usr/share/nginx/html

Копируем конфиг nginx для SPA (поддержка роутинга)

RUN echo 'server { \

listen 3000; \

location / { \

root /usr/share/nginx/html; \

index index.html index.htm; \

try_files \$uri \$uri/ /index.html; \

} \

}' > /etc/nginx/conf.d/default.conf

EXPOSE 3000

CMD ["nginx", "-g", "daemon off;"]

Текущий LectureSettingsPage.tsx:

import {

Copy,

Eye,

EyeOff,

Globe,

Loader2,

Lock,

```

    Mail,
    Play,
    QRCode,
    Save
} from 'lucide-react'
import {useCallback, useEffect, useState} from 'react'
import { useNavigate, useParams } from 'react-router'
import { toast } from 'sonner'
import {
    BASE_URL,
    getLecture,
    getSlideSequence,
    startLecture,
    updateLecture
} from '../app/api/client'
import { Tooltip, TooltipContent, TooltipTrigger } from '../shared/tooltip'

interface SlidePreview {
    index: number
    imageUrl: string
}

export function LectureSettingsPage() {
    const navigate = useNavigate()
    const { lectureId } = useParams<{ lectureId: string }>()
    const [lectureName, setLectureName] = useState("")
    const [description, setDescription] = useState("")
    const [slidePreviews, setSlidePreviews] = useState<SlidePreview[]>([])
    const [loadingLecture, setLoadingLecture] = useState(true)
    const [saving, setSaving] = useState(false)

    const [startSlide, setStartSlide] = useState('1')
    const [accessType, setAccessType] = useState<
        'open' | 'password' | 'invitation'
    >('open')
    const [password, setPassword] = useState("")
    const [showPassword, setShowPassword] = useState(false)
    const [duration, setDuration] = useState('90')
    const [allowQuestions, setAllowQuestions] = useState(true)
    const [showQR, setShowQR] = useState(false)

    // Состояние для отслеживания изменений
    const [hasUnsavedChanges, setHasUnsavedChanges] = useState(false)

```

```
const [showUnsavedDialog, setShowUnsavedDialog] = useState(false)
const [pendingStart, setPendingStart] = useState(false)
```

```
// Сохраняем исходные значения при загрузке
const [initialValues, setInitialValues] = useState({
  lectureName: "",
  description: "",
  accessType: 'open' as 'open' | 'password' | 'invitation',
  password: "",
  duration: '90',
  allowQuestions: true
})
```

```
// LectureSettingsPage.tsx - добавь useEffect для отслеживания изменений
```

```
// Функция проверки наличия изменений
const checkUnsavedChanges = useCallback(() => {
  const hasChanges =
    lectureName !== initialValues.lectureName ||
    description !== initialValues.description ||
    accessType !== initialValues.accessType ||
    password !== initialValues.password ||
    duration !== initialValues.duration ||
    allowQuestions !== initialValues.allowQuestions

  setHasUnsavedChanges(hasChanges)
  return hasChanges
}, [lectureName, description, accessType, password, duration, allowQuestions, initialValues])
```

```
// Следим за изменениями всех полей
useEffect(() => {
  checkUnsavedChanges()
}, [lectureName, description, accessType, password, duration, allowQuestions,
checkUnsavedChanges])
```

```
// При загрузке данных устанавливаем initialValues
useEffect(() => {
  if (!loadingLecture && lectureId && lectureId !== 'new') {
    setInitialValues({
      lectureName,
      description,
```

```

        accessType,
        password,
        duration,
        allowQuestions
    })
}
}, [loadingLecture, lectureId])

```

```

useEffect(() => {
    const handleBeforeUnload = (e: BeforeUnloadEvent) => {
        if (hasUnsavedChanges) {
            e.preventDefault()
            e.returnValue = 'У вас есть несохранённые изменения. Вы уверены, что хотите покинуть страницу?'
            return e.returnValue
        }
    }
}

window.addEventListener('beforeunload', handleBeforeUnload)
return () => window.removeEventListener('beforeunload', handleBeforeUnload)
}, [hasUnsavedChanges])

```

```

// Load lecture + restore access settings from localStorage
useEffect(() => {
    if (!lectureId || lectureId === 'new') {
        setLoadingLecture(false)
    }
}

```

```

    return
  }

  // Restore access settings saved locally
  try {
    const saved = localStorage.getItem(`lecture_access_${lectureId}`)
    if (saved) {
      const parsed = JSON.parse(saved)
      if (parsed.accessType) setAccessType(parsed.accessType)
      if (parsed.password) setPassword(parsed.password)
      if (parsed.duration) setDuration(parsed.duration)
      if (parsed.allowQuestions !== undefined)
        setAllowQuestions(parsed.allowQuestions)
    }
  } catch {}

  ;(async () => {
    try {
      setLoadingLecture(true)
      const lecture = await getLecture(parseInt(lectureId))
      setLectureName(lecture.name || '')

      if (lecture.sequenceId) {
        const seq = await getSlideSequence(lecture.sequenceId)
        const slides: SlidePreview[] = (seq.slides || []).map(
          (_, idx: number) => ({
            index: idx + 1,
            imageUrl: `${BASE_URL}/slide-sequences/${lecture.sequenceId}/slide/${idx + 1}`
          })
        )
        setSlidePreviews(slides)
        setStartSlide('1')
      }
    } catch (e) {
      toast.error('Не удалось загрузить лекцию')
    } finally {
      setLoadingLecture(false)
    }
  })()
}, [lectureId])

// Студенты подключаются через Telegram-бота командой /join
const BOT_USERNAME = 'lecturer_assistant_bot'
```

```
const telegramLink = `https://t.me/${BOT_USERNAME}?start=join_${lectureId}`  
const joinCommand = `/join ${lectureName} | | lectureId`  
const qrUrl = `https://api.qrserver.com/v1/create-qr-  
code/?size=200x200&data=${encodeURIComponent(telegramLink)}`
```

// LectureSettingsPage.tsx - обнови существующую функцию handleSave

```
const handleSave = async () => {  
  if (!lectureName.trim()) {  
    toast.error('Введите название лекции')  
    return false  
  }  
  if (accessType === 'password' && !password.trim()) {  
    toast.error('Введите пароль')  
    return false  
  }  
  if (!lectureId) return false  
  
  // Save access settings locally  
  localStorage.setItem(  
    `lecture_access_${lectureId}`,  
    JSON.stringify({  
      accessType,  
      password,  
      duration,  
      allowQuestions  
    })  
  )  
  
  try {  
    setSaving(true)  
    await updateLecture(parseInt(lectureId), {  
      name: lectureName.trim(),  
      accessType: accessType.toUpperCase(),  
      password: accessType === 'password' ? password.trim() : ""  
    })  
  }  
  
  // После успешного сохранения обновляем initialValues  
  setInitialValues({  
    lectureName,  
    description,  
    accessType,  
    password,
```

```

        duration,
        allowQuestions
    })
    setHasUnsavedChanges(false)

    toast.success('Настройки сохранены')
    return true
  } catch (e) {
    toast.error('Ошибка при сохранении')
    return false
  } finally {
    setSaving(false)
  }
}

```

// LectureSettingsPage.tsx - измени существующую функцию handleStart

```

const handleStart = async () => {
  if (!lectureName.trim()) {
    toast.error('Сначала заполните название')
    return
  }
  if (accessType === 'password' && !password.trim()) {
    toast.error('Задайте пароль для лекции')
    return
  }
  if (!lectureId) {
    toast.error('ID лекции не найден')
    return
  }

  // Проверяем наличие несохранённых изменений
  if (hasUnsavedChanges) {
    setPendingStart(true)
    setShowUnsavedDialog(true)
    return
  }

  // Если изменений нет - запускаем сразу
  await executeStart()
}

```

// Выносим логику запуска в отдельную функцию

```

const executeStart = async () => {
  if (!lectureId) return

  try {
    await startLecture(parseInt(lectureId))
    toast.success('Лекция запущена!')
    navigate(`/live/${lectureId}`)
  } catch (error) {
    console.error('Failed to start lecture:', error)
    toast.error('Ошибка при запуске лекции')
  }
}

const copyToClipboard = (text: string) => {
  navigator.clipboard.writeText(text).then(() => toast.success('Скопировано'))
}

const Toggle = ({
  value,
  onChange
}: {
  value: boolean
  onChange: () => void
}) => (
  <button
    onClick={onChange}
    className={`w-10 h-6 rounded-full transition-colors relative flex-shrink-0 ${value ? 'bg-orange-500' : 'bg-neutral-300'}`}
    >
    <div
      className={`w-4 h-4 bg-white rounded-full absolute top-1 transition-all ${value ? 'left-5' : 'left-1'}`}
    />
  </button>
)

if (loadingLecture) {
  return (
    <div className="p-4 sm:p-6 lg:p-8 flex items-center justify-center py-24">
      <Loader2 className="w-8 h-8 text-orange-500 animate-spin" />
    </div>
  )
}

```

```

// Модальное окно подтверждения
const UnsavedChangesDialog = () => {
  if (!showUnsavedDialog) return null

  const handleSaveAndStart = async () => {
    setShowUnsavedDialog(false)
    const saveSuccess = await handleSave()

    if (saveSuccess && pendingStart) {
      await executeStart()
    }
    setPendingStart(false)
  }

  const handleStartWithoutSave = async () => {
    setShowUnsavedDialog(false)
    if (pendingStart) {
      await executeStart()
    }
    setPendingStart(false)
  }

  const handleCancel = () => {
    setShowUnsavedDialog(false)
    setPendingStart(false)
  }

  return (
    <div className="fixed inset-0 bg-black/50 z-50 flex items-center justify-center p-4">
      <div className="bg-white rounded-xl shadow-xl max-w-md w-full">
        <div className="p-6 border-b border-neutral-200">
          <h2 className="text-xl font-semibold text-neutral-900">
            Несохранённые изменения
          </h2>
          <p className="text-sm text-neutral-500 mt-1">
            В настройках лекции есть несохранённые изменения
          </p>
        </div>

        <div className="p-6">

```

```

<div className="flex items-start gap-3 mb-4 p-3 bg-amber-50 rounded-lg border
border-amber-200">
  <div className="flex-shrink-0">
    <svg className="w-5 h-5 text-amber-600" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M12 9v2m0 4h.01m-6.938 4h13.856c1.54 0 2.502-1.667 1.732-3L13.732 4c-.77-1.333-
2.694-1.333-3.464 0L3.34 16c-.77 1.333.192 3 1.732 3z" />
    </svg>
  </div>
  <div className="flex-1">
    <p className="text-sm text-neutral-700 font-medium">
      Вы хотите сохранить изменения перед запуском?
    </p>
    <p className="text-xs text-neutral-500 mt-1">
      Если не сохранить, все внесённые изменения будут потеряны
    </p>
  </div>
</div>

<div className="p-6 border-t border-neutral-200 flex gap-3">
  <button
    onClick={handleCancel}
    className="flex-1 px-4 py-2 border border-neutral-300 rounded-lg text-neutral-
700 hover:bg-neutral-50 transition-colors"
  >
    Отмена
  </button>
  <button
    onClick={handleSaveAndStart}
    disabled={saving}
    className="flex-1 px-4 py-2 bg-orange-500 text-white rounded-lg hover:bg-
orange-600 transition-colors flex items-center justify-center gap-2 disabled:opacity-50"
  >
    {saving ? <Loader2 className="w-4 h-4 animate-spin" /> : <Save
className="w-4 h-4" />}
    Сохранить и запустить
  </button>
  <button
    onClick={handleStartWithoutSave}
    className="flex-1 px-4 py-2 bg-green-600 text-white rounded-lg hover:bg-
green-700 transition-colors flex items-center justify-center gap-2"

```

```

        >
        <Play className="w-4 h-4" />
        Запустить без сохранения
    </button>
</div>
</div>
</div>
)
}

```

```

return (
  <div className="p-4 sm:p-6 lg:p-8">
    <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4 mb-6">
      <div>
        <h1 className="mb-1">Настройка лекции</h1>
        <p className="text-sm text-neutral-500">
          Настройте параметры перед запуском
        </p>
      </div>
      <div className="flex gap-2">
        <Tooltip>
          <TooltipTrigger asChild>
            <button
              onClick={handleSave}
              disabled={saving}
              className="flex items-center gap-2 px-4 py-2 border border-neutral-300 rounded-lg hover:bg-neutral-50 text-sm disabled:opacity-50"
            >
              {saving ? (
                <Loader2 className="w-4 h-4 animate-spin" />
              ) : (
                <Save className="w-4 h-4" />
              )}
              Сохранить
            </button>
          </TooltipTrigger>
          <TooltipContent>
            <p>Сохранить изменения лекции</p>
          </TooltipContent>
        </Tooltip>
      </div>
    </div>
  </div>
)

```

```

        </TooltipContent>
      </Tooltip>
    <Tooltip>
      <TooltipTrigger asChild>
        <button
          onClick={handleStart}
          className="flex items-center gap-2 px-4 py-2 bg-green-600 text-white rounded-lg
hover:bg-green-700 text-sm"
        >
          <Play className="w-4 h-4" /> Начать лекцию
        </button>
      </TooltipTrigger>
      <TooltipContent>
        <p>Запустить лекцию для студентов</p>
      </TooltipContent>
    </Tooltip>
  </div>
</div>

```

```

<div className="grid grid-cols-1 lg:grid-cols-[1fr,380px] gap-6">
  { /* Left */ }
  <div className="space-y-6">
    <div className="bg-white rounded-xl p-5 border border-neutral-200">
      <h3 className="text-sm mb-4">Данные лекции</h3>
      <div className="mb-4">
        <label className="block text-sm mb-1.5">Название</label>
        <input
          type="text"
          value={lectureName}
          onChange={e => setLectureName(e.target.value)}
          className="w-full px-4 py-2.5 bg-neutral-50 border border-neutral-300 rounded-
lg focus:outline-none focus:ring-2 focus:ring-orange-500"
        />
      </div>
      <div className="mb-4">
        <label className="block text-sm mb-1.5">Описание</label>
        <textarea
          value={description}
          onChange={e => setDescription(e.target.value)}
          rows={3}
          placeholder="Краткое описание темы..."
          className="w-full px-4 py-2.5 bg-neutral-50 border border-neutral-300 rounded-
lg focus:outline-none focus:ring-2 focus:ring-orange-500 resize-none"

```

```

    />
  </div>
</div>

{/* Real slide previews */}
{slidePreviews.length > 0 && (
  <div className="bg-white rounded-xl p-5 border border-neutral-200">
    <h3 className="text-sm mb-4">Слайды ({slidePreviews.length})</h3>
    <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
      {slidePreviews.slice(0, 8).map(s => (
        <div
          key={s.index}
          className="aspect-video bg-neutral-100 rounded-lg overflow-hidden"
        >
          <img
            src={s.imageUrl}
            alt={`Слайд ${s.index}`}
            className="w-full h-full object-cover"
          />
        </div>
      ))}
    </div>
    {slidePreviews.length > 8 && (
      <p className="text-xs text-neutral-500 mt-2">
        и ещё {slidePreviews.length - 8} слайдов...
      </p>
    )}
  </div>
)}

{/* Access & QR */}
<div className="bg-white rounded-xl p-5 border border-neutral-200">
  <h3 className="text-sm mb-4">Доступ к лекции</h3>
  <div className="space-y-3 mb-4">
    {[
      {
        value: 'open' as const,
        label: 'Открытый доступ',
        desc: 'Любой студент по ссылке',
        icon: Globe
      },
      {
        value: 'password' as const,

```

```

      label: 'Защита паролем',
      desc: 'Студенты вводят пароль при подключении',
      icon: Lock
    },
    {
      value: 'invitation' as const,
      label: 'Только по приглашению',
      desc: 'Только по QR-коду или прямой ссылке',
      icon: Mail
    }
  ].map(opt => (
    <label
      key={opt.value}
      className={`flex items-start gap-3 p-3 rounded-lg border-2 cursor-pointer
transition-colors ${
        accessType === opt.value
          ? 'border-orange-500 bg-orange-50'
          : 'border-neutral-200 hover:border-neutral-300'
      }}
    >
      <input
        type="radio"
        name="access"
        value={opt.value}
        checked={accessType === opt.value}
        onChange={() => setAccessType(opt.value)}
        className="w-4 h-4 accent-orange-500 mt-0.5"
      />
      <div className="flex-1">
        <div className="flex items-center gap-2 text-sm">
          <opt.icon className="w-4 h-4 text-neutral-500" />{' '}
          {opt.label}
        </div>
        <div className="text-xs text-neutral-500 mt-0.5">
          {opt.desc}
        </div>
      </div>
    </label>
  )))
</div>

{accessType === 'password' && (
  <div className="p-4 bg-orange-50 border border-orange-200 rounded-lg">

```

```

<label className="block text-sm mb-1.5">
  Пароль для подключения
</label>
<div className="flex gap-2">
  <div className="relative flex-1">
    <input
      type={showPassword ? 'text' : 'password'}
      value={password}
      onChange={e => setPassword(e.target.value)}
      className="w-full px-4 py-2.5 pr-10 bg-white border border-orange-300
rounded-lg focus:outline-none focus:ring-2 focus:ring-orange-500"
    />
    <Tooltip>
      <TooltipTrigger asChild>
        <button
          onClick={() => setShowPassword(!showPassword)}
          className="absolute right-3 top-1/2 -translate-y-1/2 text-neutral-400
hover:text-neutral-600"
        >
          {showPassword ? (
            <EyeOff className="w-4 h-4" />
          ) : (
            <Eye className="w-4 h-4" />
          )}
        </button>
      </TooltipTrigger>
      <TooltipContent>
        <p>
          {showPassword ? 'Скрыть пароль' : 'Показать пароль'}
        </p>
      </TooltipContent>
    </Tooltip>
  </div>
  <Tooltip>
    <TooltipTrigger asChild>
      <button
        onClick={() => copyToClipboard(password)}
        className="px-3 py-2 border border-orange-300 rounded-lg hover:bg-
orange-100 text-sm flex items-center gap-1"
      >
        <Copy className="w-3.5 h-3.5" /> Копировать
      </button>
    </TooltipTrigger>
  </Tooltip>

```

```

        <TooltipContent>
          <p>Копировать пароль в буфер обмена</p>
        </TooltipContent>
      </Tooltip>
    </div>
    {password && (
      <div className="mt-2 flex items-center gap-2 text-sm">
        <span className="text-neutral-600">Пароль:</span>
        <code className="bg-white px-2 py-0.5 rounded border border-orange-200
text-orange-700">
          {password}
        </code>
      </div>
    )}
  </div>
)}

{accessType === 'invitation' && (
  <div className="p-4 bg-orange-50 border border-orange-200 rounded-lg space-y-
3">
    <div>
      <label className="block text-sm mb-1.5">
        Команда для студентов (Telegram)
      </label>
      <div className="flex gap-2">
        <code className="flex-1 px-3 py-2 bg-white border border-orange-300
rounded-lg text-sm text-orange-800 font-mono">
          {joinCommand}
        </code>
        <Tooltip>
          <TooltipTrigger asChild>
            <button
              onClick={() => copyToClipboard(joinCommand)}
              className="px-3 py-2 border border-orange-300 rounded-lg hover:bg-
orange-100 text-sm flex items-center gap-1"
            >
              <Copy className="w-3.5 h-3.5" />
            </button>
          </TooltipTrigger>
          <TooltipContent>
            <p>Копировать команду для подключения</p>
          </TooltipContent>
        </Tooltip>
      </div>
    </div>
  )}
)

```

```

    </div>
    <p className="text-xs text-neutral-500 mt-1">
      Студент отправляет эту команду боту{' '}
      <span className="font-medium">@{BOT_USERNAME}</span>
    </p>
  </div>
  <div>
    <div className="flex items-center justify-between mb-2">
      <label className="text-sm">
        QR-код → открывает Telegram-бота
      </label>
      <Tooltip>
        <TooltipTrigger asChild>
          <button
            onClick={() => setShowQR(!showQR)}
            className="flex items-center gap-1 text-sm text-orange-600 hover:text-
orange-700"
          >
            <QrCode className="w-4 h-4" />{' '}
            {showQR ? 'Скрыть' : 'Показать QR'}
          </button>
        </TooltipTrigger>
        <TooltipContent>
          <p>Показать QR-код для подключения</p>
        </TooltipContent>
      </Tooltip>
    </div>
    {showQR && (
      <div className="flex justify-center p-4 bg-white rounded-lg border border-
orange-200">
        <img
          src={qrUrl}
          alt="QR Code"
          className="w-48 h-48"
        />
      </div>
    )}
  </div>
</div>
</div>

```

```

    { /* Right */ }
    <div className="space-y-6">
      <div className="bg-white rounded-xl p-5 border border-neutral-200">
        <h3 className="text-sm mb-4">Параметры</h3>
        {slidePreviews.length > 0 && (
          <div className="mb-4">
            <label className="block text-sm mb-1.5">Начать со слайда</label>
            <select
              value={startSlide}
              onChange={e => setStartSlide(e.target.value)}
              className="w-full px-4 py-2.5 bg-neutral-50 border border-neutral-300
rounded-lg focus:outline-none focus:ring-2 focus:ring-orange-500"
            >
              {slidePreviews.map(s => (
                <option
                  key={s.index}
                  value={s.index}
                >
                  Слайд {s.index}
                </option>
              ))}
            </select>
          </div>
        )}
        <div className="mb-4">
          <label className="block text-sm mb-1.5">Длительность (мин)</label>
          <input
            type="number"
            value={duration}
            onChange={e => setDuration(e.target.value)}
            className="w-full px-4 py-2.5 bg-neutral-50 border border-neutral-300 rounded-
lg focus:outline-none focus:ring-2 focus:ring-orange-500"
          />
        </div>
        <div className="space-y-3 border-t border-neutral-200 pt-4">
          <div className="flex items-center justify-between">
            <span className="text-sm">Разрешить вопросы</span>
            <Toggle
              value={allowQuestions}
              onChange={() => setAllowQuestions(!allowQuestions)}
            />
          </div>
        </div>
      </div>
    </div>

```

</div>

```
{accessType === 'password' && password && (  
  <div className="bg-orange-500 text-white rounded-xl p-5">  
    <div className="flex items-center gap-2 mb-2">  
      <Lock className="w-4 h-4" />  
      <span className="text-sm">Пароль для студентов</span>  
    </div>  
    <div className="text-2xl tracking-wider mb-1">{password}</div>  
    <p className="text-orange-100 text-xs">  
      Покажите студентам при подключении  
    </p>  
  </div>  
)}
```

```
{accessType === 'invitation' && (  
  <div className="bg-orange-500 text-white rounded-xl p-5 text-center">  
    <QrCode className="w-5 h-5 mx-auto mb-2" />  
    <p className="text-sm mb-2">QR → откроет бота в Telegram</p>  
    <div className="bg-white rounded-lg p-3 inline-block mb-2">  
      <img  
        src={qrUrl}  
        alt="QR"  
        className="w-32 h-32"  
      />  
    </div>  
    <p className="text-orange-100 text-xs">  
      Выведите на проектор для студентов  
    </p>  
    <div className="mt-3 bg-orange-600 rounded-lg px-3 py-2">  
      <p className="text-orange-200 text-xs mb-1">  
        Или команда боту:  
      </p>  
      <code className="text-white text-sm font-mono">  
        {joinCommand}  
      </code>  
    </div>  
  </div>  
)}
```

</div>

</div>

{/* Модальное окно подтверждения */}

<UnsavedChangesDialog />

```
</div>  
)  
}
```